

Salesforce Login Verification Code — Full Explanation

What This Does

Every time a user logs in to Salesforce — whether through username/password or SSO via Google SAML — they are required to enter a 6-digit code sent to their registered email before accessing the org. Three consecutive wrong codes result in an automatic logout and a 10-minute block.

Architecture Overview



Components

1. Custom Object — Verification_Code__c

Stores one temporary record per login attempt. The code is never stored in plain text.

Field	Type	Purpose
User__c	Lookup → User	Which user this code belongs to
Code_Hash__c	Text(255)	SHA-256 hash of the 6-digit code
Expiry_Time__c	DateTime	5 minutes from creation
Attempt_Count__c	Number	Incremented on each wrong entry
Is_Used__c	Checkbox	True when consumed or attempts exhausted

2. Apex Classes

VerificationCodeService

Core business logic. Two public static methods:

`generateAndSendCode(userId, userEmail, userName)`

- Marks any existing active code for the user as `Is_Used__c = true`
- Generates a cryptographically random 6-digit number
- Hashes it with SHA-256 and saves a new `Verification_Code__c` record (expires in 5 minutes)
- Emails the plain code to the user
- Returns `{ isSuccess: true/false }`

`validateCode(userId, enteredCode)`

- Queries the most recent unused, non-expired code for the user
- Hashes the entered code and compares to stored hash
- Returns one of five statuses:

Status	Meaning
SUCCESS	Code correct — allow login
INVALID_CODE	Wrong code, attempts remaining
MAX_ATTEMPTS_EXCEEDED	3rd wrong code — block and logout
EXPIRED	Code older than 5 minutes — send a new one
NO_PENDING_CODE	No code found — send a new one

`VerificationCodeGenerator`

Thin `@InvocableMethod` wrapper so the Flow can call `generateAndSendCode`.

`VerificationCodeValidator`

Thin `@InvocableMethod` wrapper so the Flow can call `validateCode`.

`VerificationCodeChecker`

Called at the **very start** of every login flow run — before any code is generated.

Queries for any `Verification_Code__c` record where:

- `User__c = current user`
- `Attempt_Count__c >= 3`
- `Is_Used__c = true`
- `CreatedDate >= now - 10 minutes`

Returns `{ isBlocked: true/false }`.

This is the **refresh guard**. Without it, refreshing the browser during a blocked state would restart the flow and generate a fresh code, resetting the attempt counter.

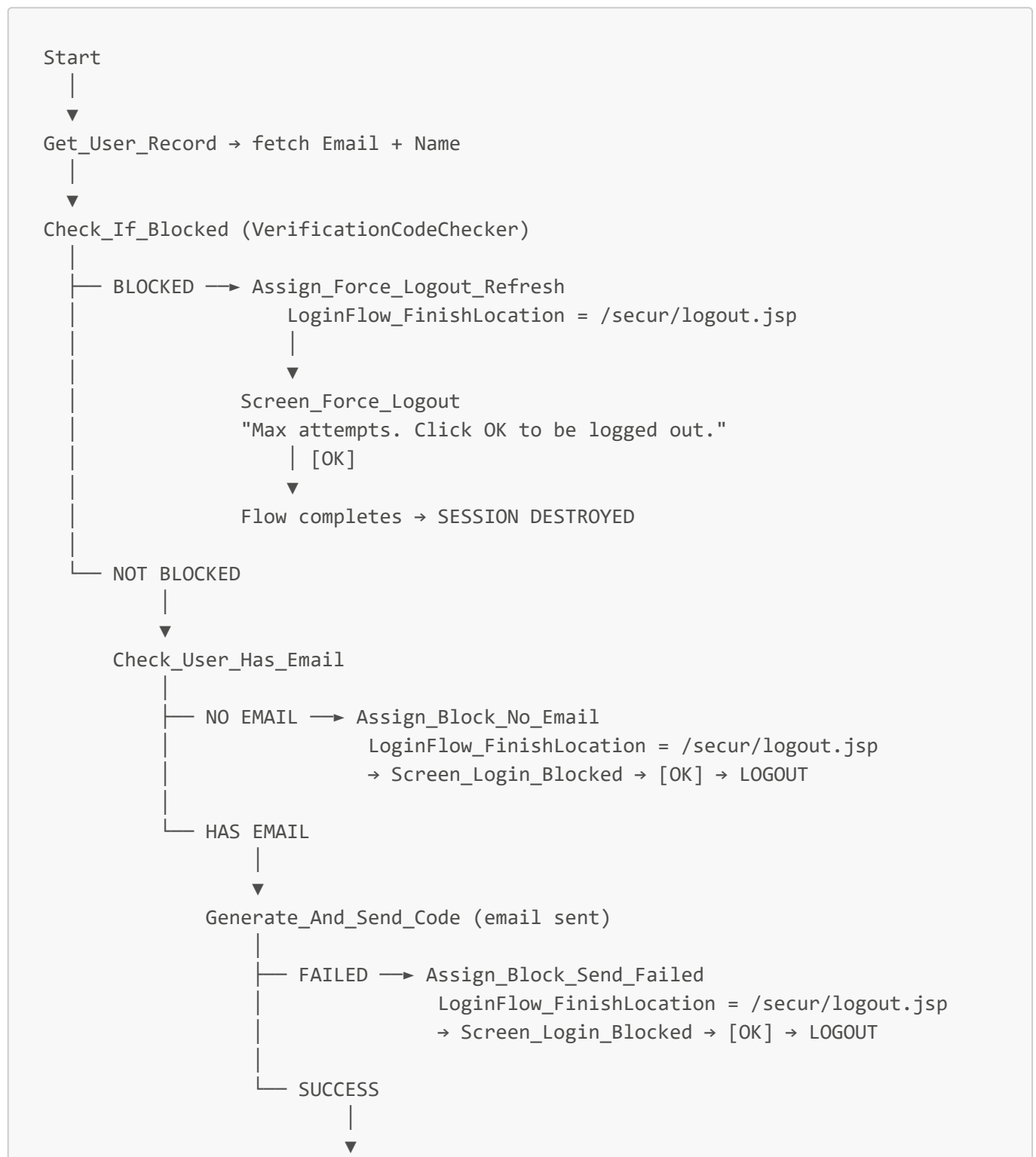
VerificationCodeCleanup

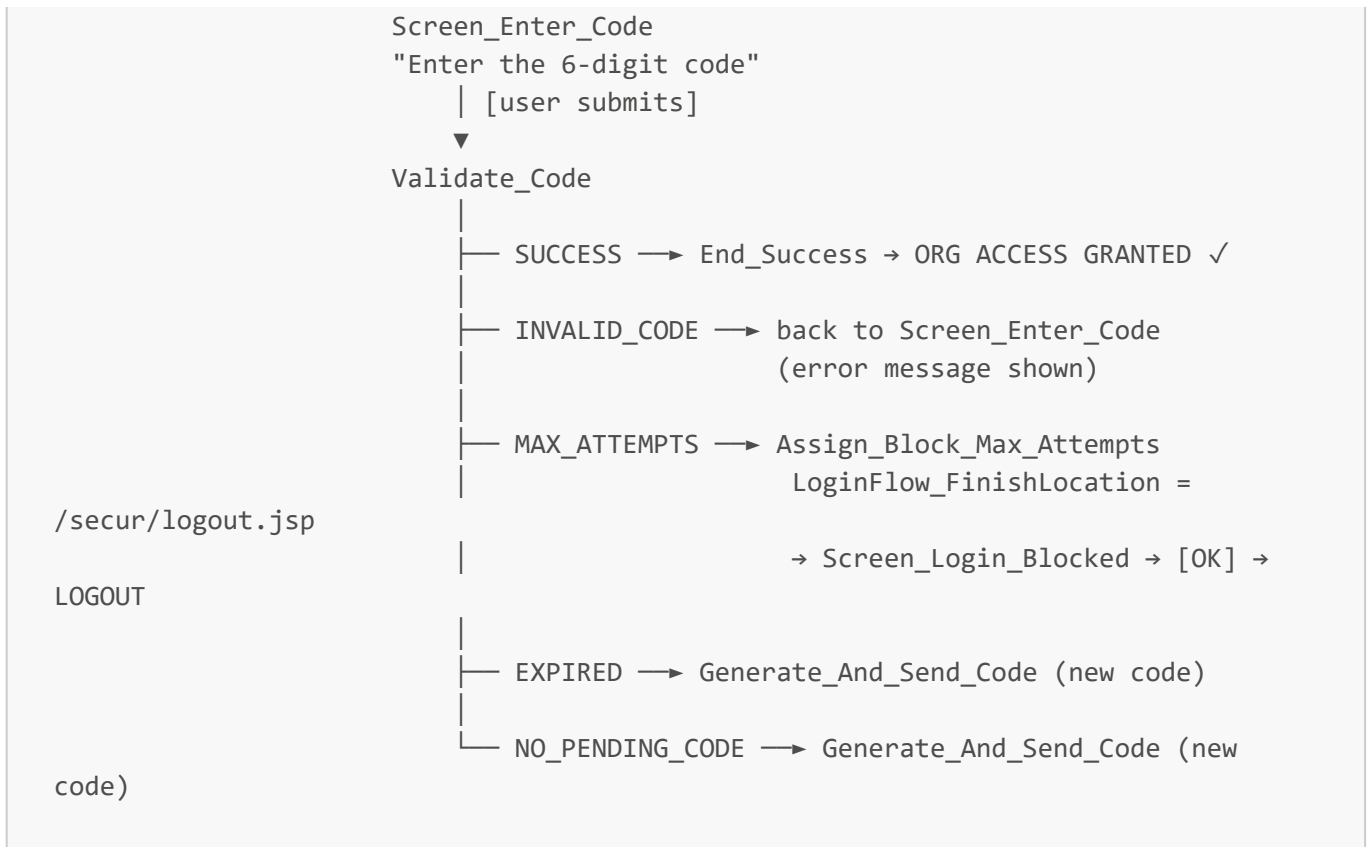
A `Schedulable` class that deletes `Verification_Code__c` records older than 24 hours. Scheduled to run daily at 2 AM.

3. Login Flow — `VerificationCodeLoginFlow`

A Salesforce Screen Flow assigned as a Login Flow. Runs after authentication but **before org access is granted**.

Full Flow Logic





How Blocking Works

The Block Mechanism

There is **no separate "blocked" flag or field**. The block is inferred at runtime by `VerificationCodeChecker`.

When a user enters their 3rd wrong code:

1. `VerificationCodeService.validateCode` increments `Attempt_Count__c` to 3 and sets `Is_Used__c = true`
2. Returns `MAX_ATTEMPTS_EXCEEDED` to the flow
3. Flow sets `LoginFlow_FinishLocation = /secur/logout.jsp` and shows the blocked screen
4. User clicks OK → flow completes → **session destroyed**

Why the Block Holds on Refresh

On every new login, `Check_If_Blocked` runs BEFORE any code is generated:

```

SELECT Id FROM Verification_Code__c
WHERE User__c = :userId
AND Attempt_Count__c >= 3
AND Is_Used__c = true
AND CreatedDate >= :DateTime.now().addMinutes(-10)
LIMIT 1

```

If this finds a record → user is blocked → flow routes to the force-logout screen. No new code is generated, no new attempts are given.

How Long Is the Block?

10 minutes — controlled by one constant in `VerificationCodeChecker`:

```
@TestVisible public static final Integer BLOCK_DURATION_MINUTES = 10;
```

To change the duration, update this constant and redeploy the class. No object, flow, or other changes needed.

The Key Security Variable: `LoginFlow_FinishLocation`

This is Salesforce's built-in Login Flow output variable. When the flow finishes:

- **Empty / null** → user gets normal org access
- `/secur/logout.jsp` → Salesforce destroys the server-side session and redirects to `login.salesforce.com`

Every blocking assignment in the flow sets `LoginFlow_FinishLocation = /secur/logout.jsp` **before** routing to the blocked screen. This means no matter what the user does — clicks OK, or finds any other way to complete the flow — they **cannot reach the org**.

Security Design Decisions

Decision	Reason
Codes hashed with SHA-256	Plain codes never stored in the database
<code>WITHOUT SHARING</code> on Apex classes	Must run in system context during Login Flow
<code>LoginFlow_FinishLocation</code> set on ALL blocked paths	Clicking OK on any error screen destroys the session — org access impossible
Block checked at flow START before code generation	Refresh cannot reset the attempt counter by getting a new code
Codes expire in 5 minutes	Limits window for interception
Old codes marked <code>Is_Used__c = true</code> when a new one is generated	Previous codes can never be replayed
Works for SSO and standard login	Assigned at the Login Flow level, not per auth method

Test Coverage

Class	Tests	Result
-------	-------	--------

Class	Tests	Result
VerificationCodeServiceTest	13	☒ Pass
VerificationCodeCheckerTest	4	☒ Pass
VerificationCodeCleanupTest	2	☒ Pass
Total	19	100%

Deployment Checklist

After deploying `VerificationCodeLoginFlow_Deploy.zip` via Workbench, complete these manual steps:

- 1. **Setup** → **Flows** — confirm `Verification Code Login Flow` status is **Active**
- 2. **Setup** → **Identity** → **Login Flows** → **New** — assign flow to standard login page
- 3. **Setup** → **Single Sign-On Settings** → **Edit** — assign flow to Google SAML config
- 4. **Developer Console** → **Execute Anonymous** — schedule the cleanup job:

```
System.schedule('VerificationCodeCleanup Daily', '0 0 2 * * ?', new VerificationCodeCleanup());
```